CSHARP-PDF / SAMPLE 28

Measured. Named. Mapped.

Benchmark results drawn as vector charts, brand directions for a project that does not have a final name yet, and an honest map of ISO 32000-2 parity.

- **2.50–4.14×**
  open vs PdfPig, 6 documents
- **4.55×**
  AOT fresh 240-dpi render, editor
- **15 / 0**
  Partial / Todo spec rows



01

Benchmarks

Library vs PdfPig, renderer stages, editor before / after, AOT, worker pool.

02

Brand ideas

Marks, names, palettes with contrast ratios, type pairings, mock applications.

03

Spec parity

ISO 32000-2 area by area: Done, Partial, and what is still open.

Exploratory edition. Figures from checked-in reports dated 2026-10-30. Built with this repository's own PDF writer.

Read this first

Three questions, each answered with figures that can be traced to a file in this repository: how fast is it, what might it look like, and how much of PDF 2.0 does it cover?

2.50–4.14×
faster to open than PdfPig 0.1.16, across six documents (JIT, means)

1.32–5.33×
faster to open and extract all text; PdfPig allocates less on one of six

4.55×
faster fresh 240-dpi page in the editor with AOT (3504 → 770 ms p50)

15 / 0
Partial / Todo rows in the ISO 32000-2 matrix, plus two limited Done rows

WHERE IT FALLS SHORT

Not good enough yet: the large fresh 240-dpi render did not improve in the first checkpoint (3440 → 3524 ms). Six strict interaction goals were still missed in the AOT run and three in the latest integrated profile. The worker pool is opt-in.

Contents

3 Library: open and extract
4 Library: extract and allocations
5 Library: render stages
6 Editor: before and after
7 Editor: JIT vs AOT
8 Editor: goals vs reality
9 Editor: worker pool
10 Brand: names and marks
11 Brand: Caret due diligence
12 Brand: palettes and contrast
13 Brand: type pairings
14 Brand: mock applications
15 Spec parity: overview
16 Spec parity: parser
17 Spec parity: renderer, editor
18 Method and provenance

Ground rules for every page

Traceable
Each chart carries a strip naming its source file, run date, hardware and build mode.

Misses stay visible
Unmet goals are drawn in rose; slower results point right, not hidden.

No favourable rounding
Values are as reported. Ratios are computed from reported means, to two decimals.

Names are ideas
The project name is undecided. editPDF was considered and cancelled because editpdf.io exists.

Parity is not complete
Partial rows stay Partial here. Validators are heuristic, not certification.

COLOUR KEY USED IN EVERY CHART

- PortableDocument / candidate
- PdfPig / before / non-AOT
- AOT / faster
- Miss / slower
- Goal met
- Evidence strip

Opening and reading documents

The same PDF bytes go to PortableDocument and to PdfPig 0.1.16. Before timing, both must agree on page counts and decoded text. Bars compare engines inside a row, not corpora.

Open a document and read its page count

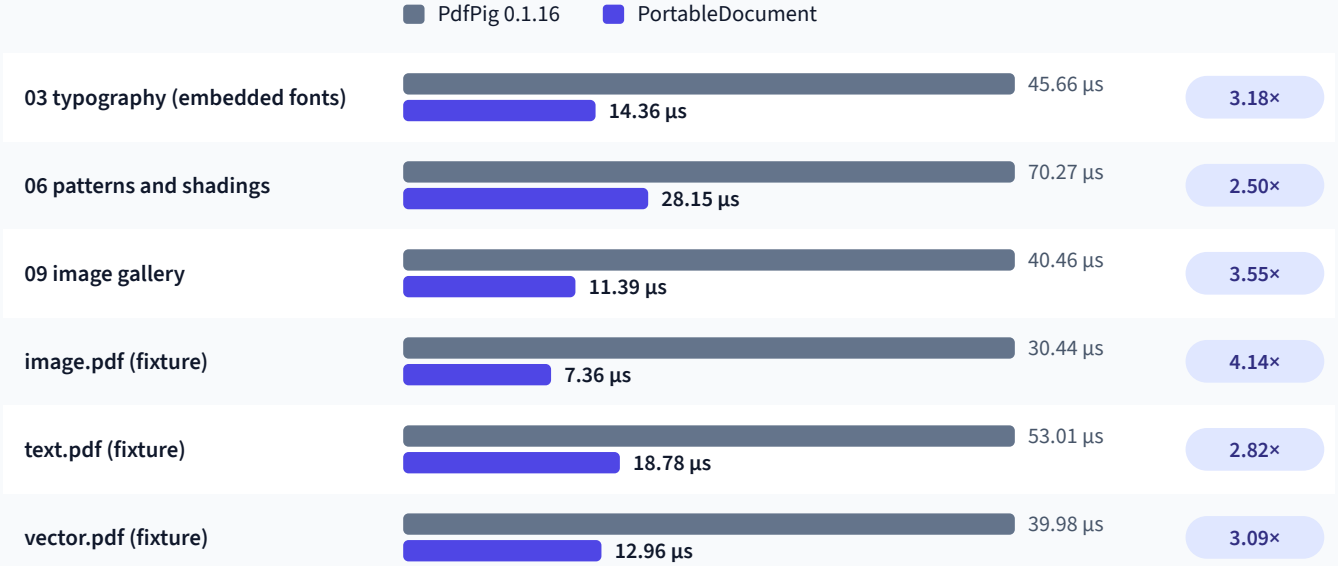


Figure 1. Mean time to open and read the page count, microseconds. Pill: PdfPig mean divided by PortableDocument mean.

Open and extract every page's text

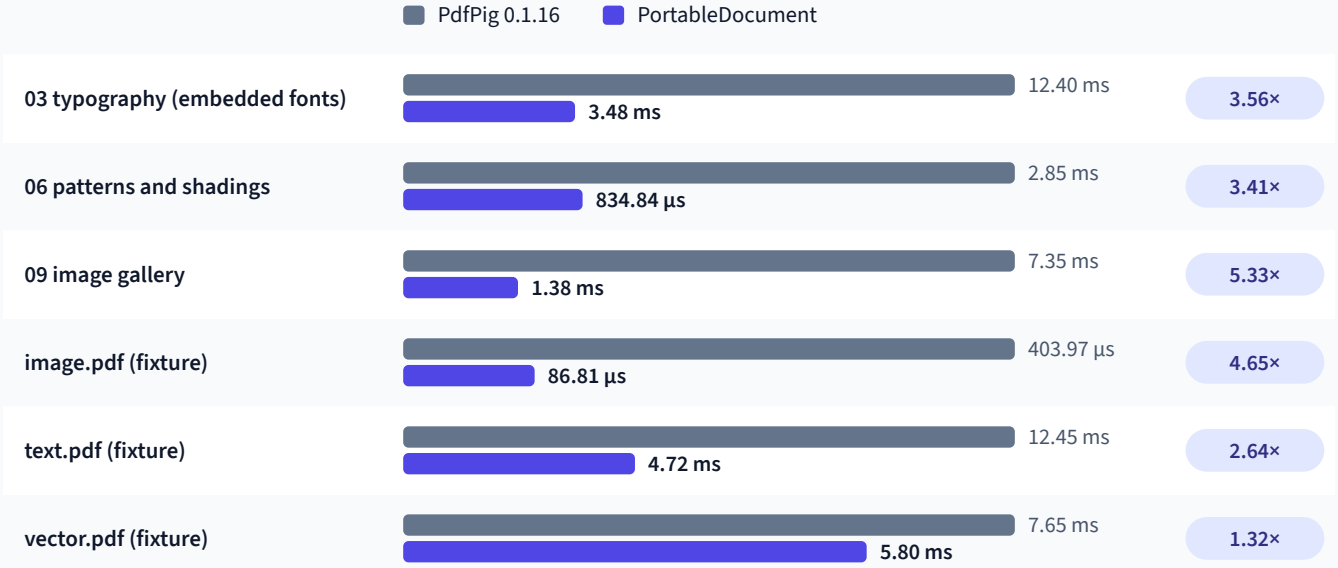


Figure 2. Mean time for a fresh open plus all-page text extraction and materialisation. The vector.pdf gap is the smallest, 1.32×.

SOURCE
bench/AdoBye.Bench/results/2026-10-30.md (PdfOpenBench, TextExtractionBench)

RUN
2026-10-30

HARDWARE
Windows 11 25H2, Intel Core i9-10900 2.80 GHz (10 cores), .NET 10.0.14 (X64 RyuJIT, JIT - not AOT), BenchmarkDotNet 0.15.8

MODE 6 warm-ups, 15 x 1 s iterations, 1 launch. Error = half-width of the 99.9% confidence interval. Same byte[] to both engines; preflight checked page counts and normalised text. Results hold for this corpus and machine only.

Reading an open document

Whiskers show the 99.9% confidence half-width BenchmarkDotNet reports. Where a whisker is wide, the ranking is weaker than the mean suggests, and the chart says so.

Extract text from a document that is already open (log scale)



Figure 3. Mean time and confidence interval per document, 32 fresh documents per iteration. Pill: PdfPig ÷ PortableDocument, from the means (BenchmarkDotNet's own Ratio column can differ).

Managed allocation for open plus extract (not total process memory)

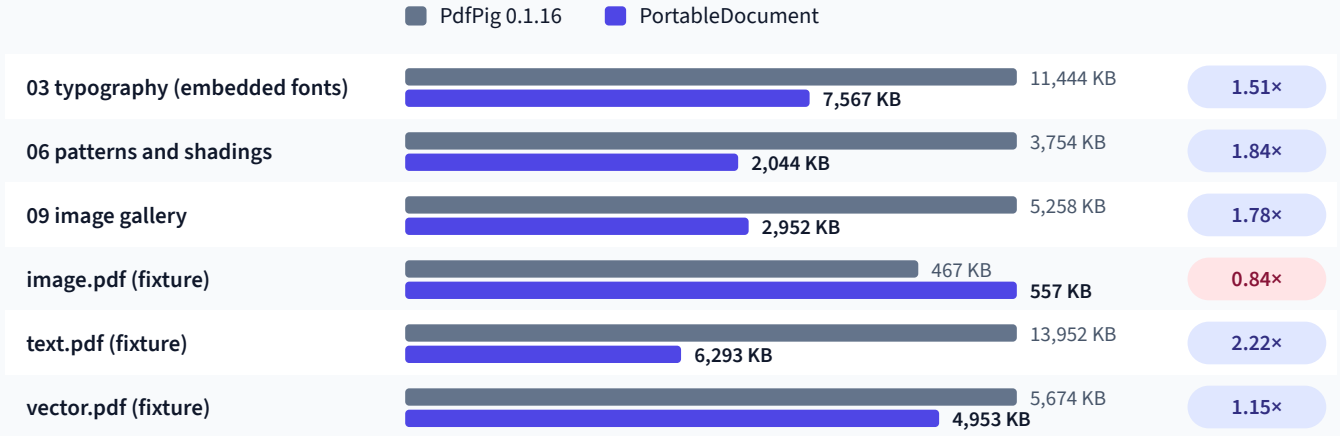


Figure 4. Rose pills mark where PdfPig allocates less than PortableDocument (image.pdf, 0.84×).

SOURCE
bench/AdoBye.Bench/results/2026-10-30.md (OpenDocumentTextBench, TextExtractionBench)

RUN
2026-10-30

HARDWARE
Windows 11 25H2, Intel Core i9-10900 2.80 GHz (10 cores), .NET 10.0.14 (X64 RyuJIT, JIT - not AOT), BenchmarkDotNet 0.15.8

MODE 6 warm-ups, 15 x 1 s iterations, 1 launch. Error = half-width of the 99.9% confidence interval. Already-open cases warn about short iterations (minimum 4.95 to 66.6 ms) and multimodal distributions; small differences are not ranked.

Where rendering time goes

The legacy render benchmark has no second engine, so this page makes no speed comparison. It shows stage costs, and why a rasteriser dominates a text page.

Stage costs on a log scale

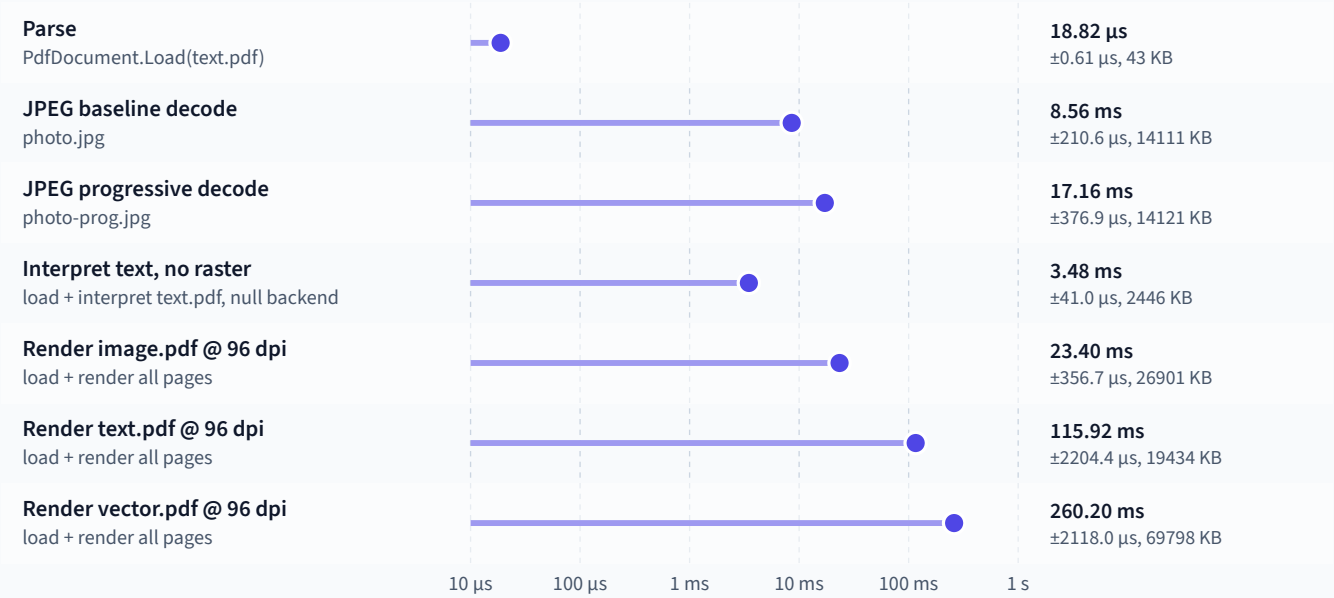


Figure 5. Mean per operation; whiskers are confidence half-widths (mostly smaller than the marker). Right: mean, ±error and managed KB.

WHAT THE NUMBERS SAY

Interpreting text.pdf with a null backend takes 3.48 ms; rendering it takes 115.9 ms. Interpretation is 3.0% of the render.

Vector rendering is 2.24× the text render and allocates 68.2 MB per operation.

Progressive JPEG decodes 2.01× slower than baseline on these photo fixtures.

WHAT THEY DO NOT SAY

No renderer comparison: PdfPig has no raster renderer in this harness.

The job label reads ShortRun, but the report shows 6 warm-ups and 15 one-second iterations.

No checked-in historical run exists, so these are a new baseline, not a speed-up.

The fixtures are small: results do not generalise to arbitrary pages.

SOURCE
bench/AdoBye.Bench/results/2026-10-30.md (RenderBench) and bench/AdoBye.Bench/Program.cs

RUN
2026-10-30

HARDWARE
Windows 11 25H2, Intel Core i9-10900 2.80 GHz (10 cores), .NET 10.0.14 (X64 RyuJIT, JIT - not AOT), BenchmarkDotNet 0.15.8

MODE 6 warm-ups, 15 x 1 s iterations, 1 launch. Error = half-width of the 99.9% confidence interval. Text/Vector/Image render every page of bench/fixtures/{text,vector,image}.pdf at 96 dpi including document load.

Editor: before and after

A bounded checkpoint, not completion of the responsiveness work. Bars to the left are faster. Three rows point right, and they are drawn that way.

Matched median latency change, non-AOT build

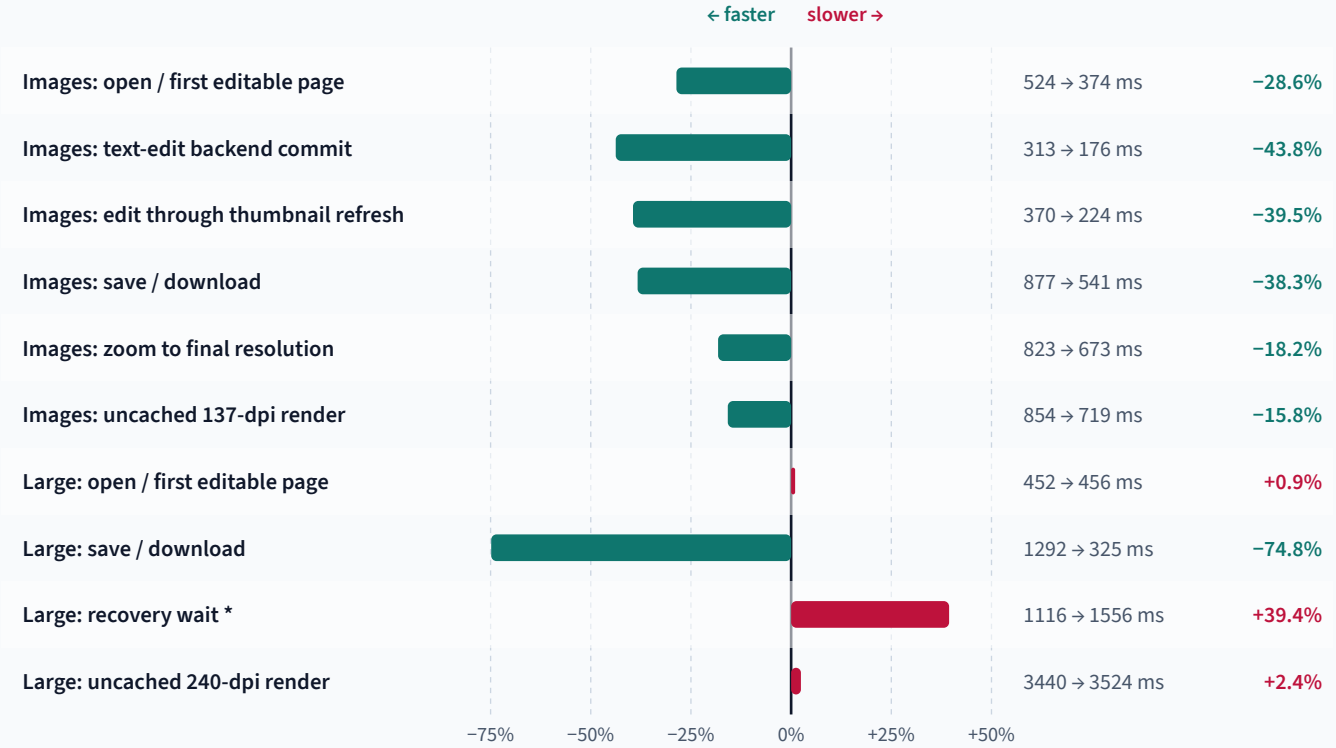


*Figure 6. * Recovery wait includes the intentional 1500 ms write-coalescing timer, so a faster download can lengthen the wait. The large fresh render is still about 3.5 s.*

Main-thread blocking (log scale)

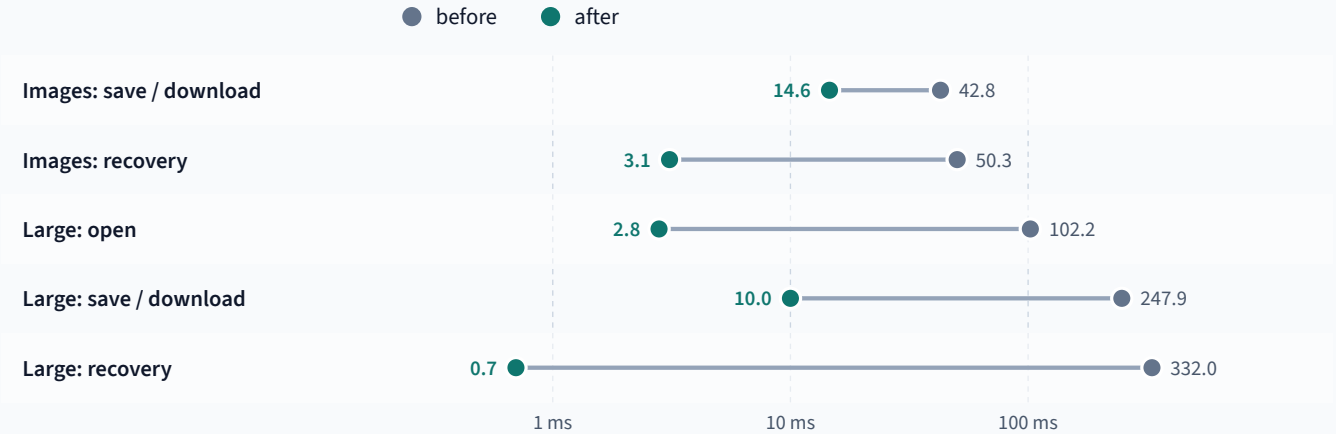


Figure 7. Maximum heartbeat gap, across three runs. This is UI blocking, not total operation time: it does not make a multi-second raster fast.

SOURCE	RUN	HARDWARE
docs/performance/2026-10-30.md	2026-10-30	Windows 11, Intel Core i9-10900, .NET 10.0.14, Playwright Chromium 1440 x 1000, device scale 1, 3 fresh contexts per run

MODE Release, local static WASM, non-AOT. Medians of three runs, four C#-generated samples plus a generated 12-page fixture. First-page PNG hashes identical before and after in every repetition. p95 is the largest of three.

Editor: JIT versus AOT

The same harness, fixtures and hardware, built twice: non-AOT (interpreter plus jiterpreter) and ahead-of-time compiled WebAssembly. Frame-bound rows barely move, and the chart shows that too.

Median latency, non-AOT versus AOT

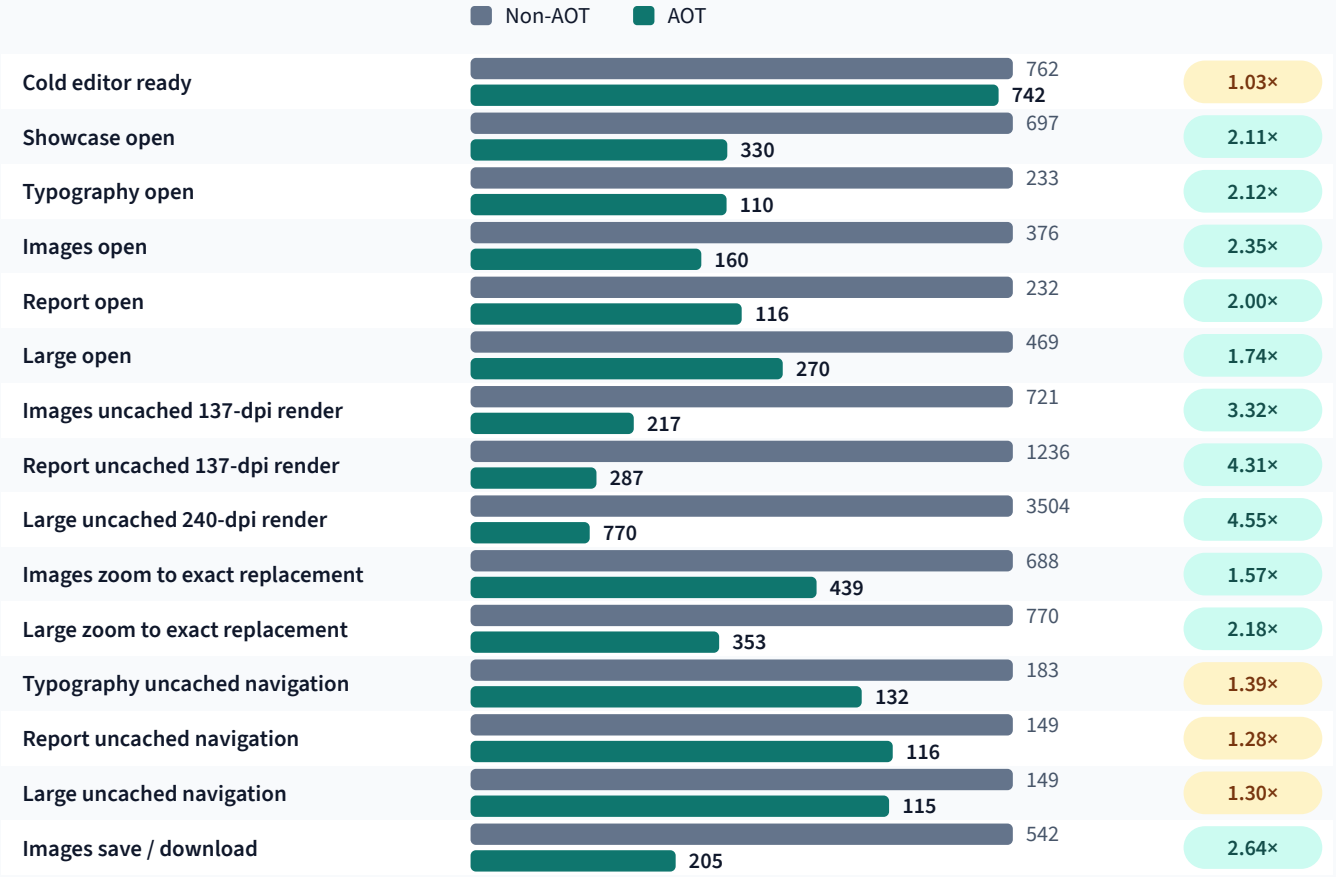


Figure 8. p50 in ms, pill = non-AOT ÷ AOT, from 1.03× (cold start, no real change) to 4.55×. In every cell p95 is within 11.5% of p50. Cached zoom (about 57–60 ms) and cached navigation (96–98 ms) are frame-bound and unchanged.

What it costs to download

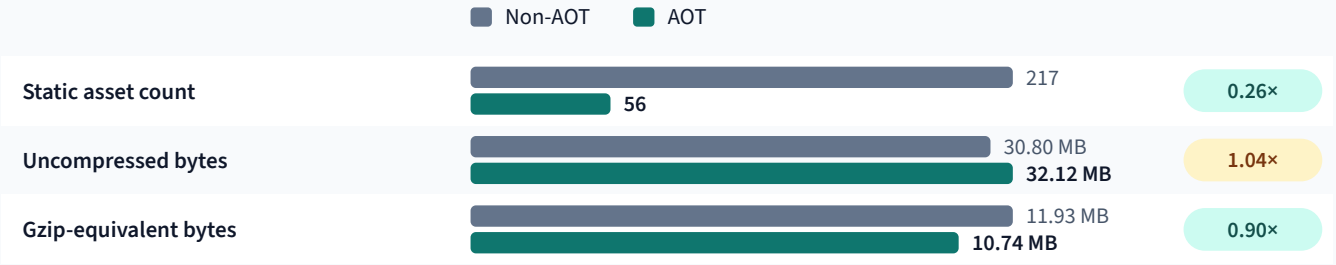


Figure 9. Pill = AOT ÷ non-AOT (below 1× is smaller). AOT is slightly larger raw. Gzip totals are estimates, not measured Pages transfer sizes.

SOURCE
docs/performance/2026-10-30-aot.md

RUN
2026-10-30

HARDWARE
Windows 11, Intel Core i9-10900, .NET 10.0.14, Playwright Chromium 1440 x 1000, device scale 1, 3 fresh contexts per run

MODE Release, unthrottled, uncompressed local server; 3 fresh contexts per variant. 15 of 15 first-page pixel hashes match the non-AOT control. Public slow-network cold start is not measured.

Editor: goals versus reality

The interaction goals are strict. Each bar is the measured p50 as a multiple of its p50 budget, so everything right of the line is a miss, and every one of them is shown.

Strict goals still missed

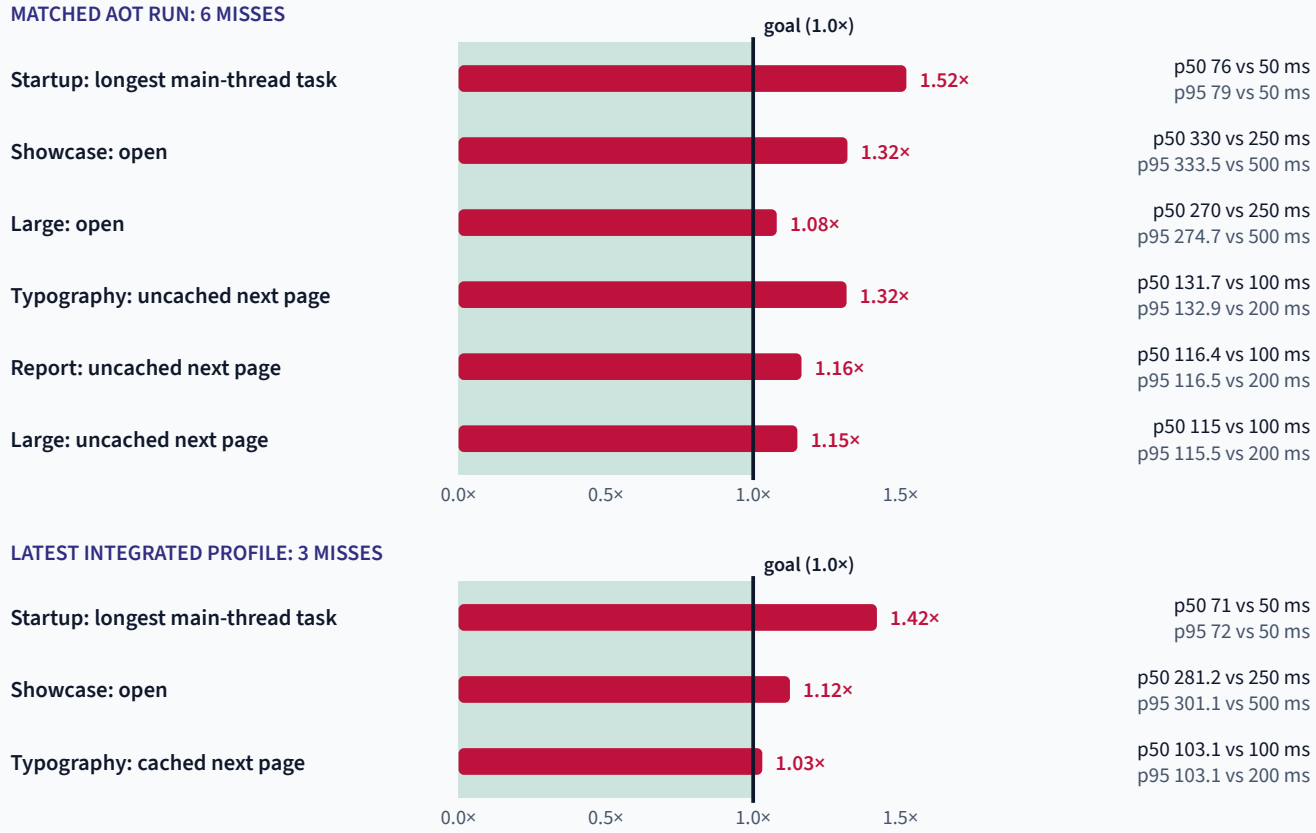


Figure 10. Green band: goal met. p50 and p95 values and budgets are printed at the right of each bar.

HOW TO READ THE MISSES

5 of the 6 AOT-run misses are p50-only; their p95 stays inside its budget. Startup misses both.

Image open (160 / 175 ms) now meets its 250 / 500 ms budget with AOT.

CI treats these as KNOWN MISSES and fails any group that exceeds 2.5× its recorded AOT p50 or p95 (50 ms minimum).

A goal met on a faster runner prints MET and never erases the recorded miss. No goal was loosened.

SOURCE
docs/performance/2026-10-30-aot-budgets.json (six) and docs/spec-matrix.md, latest integration (three)

RUN
2026-10-30

HARDWARE
Windows 11, Intel Core i9-10900, .NET 10.0.14, Playwright Chromium 1440 x 1000, device scale 1, 3 fresh contexts per run

MODE Original budgets unchanged: p50 and p95 strictly below budget. The six-miss run and the three-miss run are different profiles; do not subtract one from the other.

Editor: worker pool and the trail so far

Three render replicas split a page into bands and finish sooner in these runs. The pool stays opt-in until replicas share the fallback font assets.

One worker versus a pool of three replicas

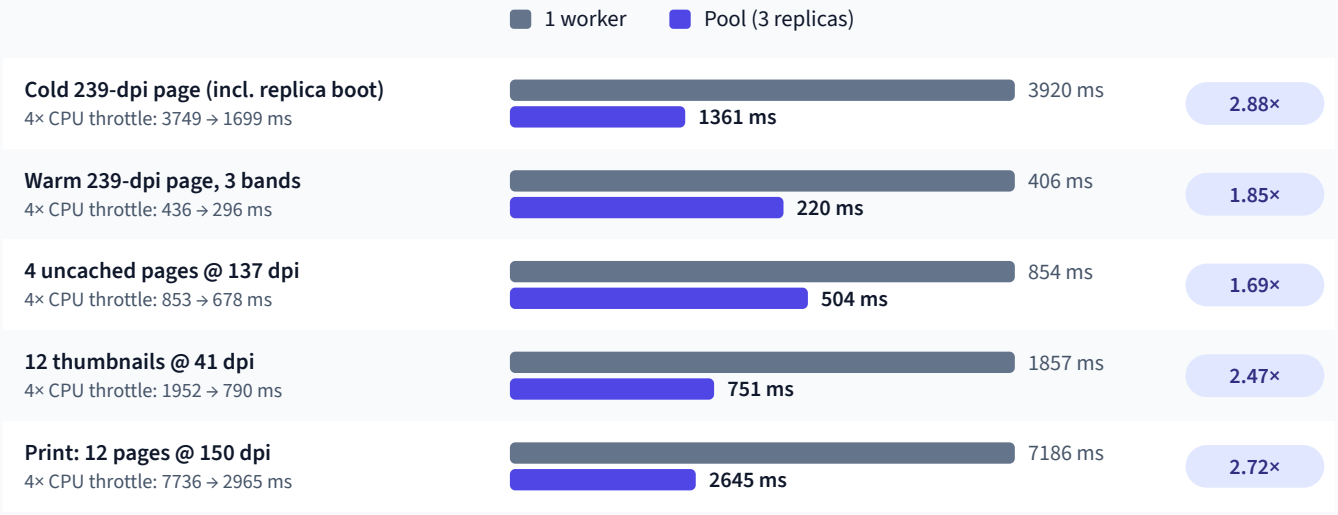


Figure 11. Decoded pixels are SHA-256-identical to the single-worker output in every scenario. Largest frame gap during the run: 79 ms (1 worker) vs 87 ms (pool), no gain. 3× was not reached for warm or navigation renders.

Before → after → AOT



Figure 12. Three separate matched runs with one harness, not one continuous series. Pill = first ÷ last. Large open was flat across the first checkpoint (452 → 456) and only moved with AOT.

SOURCE
docs/performance/2026-10-30-worker-pool.md; docs/performance/2026-10-30.md; docs/performance/2026-10-30-aot.md

RUN
2026-10-30

HARDWARE
Pool report: hardware not restated; Chromium with hardwareConcurrency forced to 4. Trails: Windows 11, Intel Core i9-10900, .NET 10.0.14, Playwright Chromium 1440 x 1000, device scale 1, 3 fresh contexts per run

MODE
Pool: non-AOT Release publish, fixture is 20-report.pdf plus six 09-images pages (12 pages). AOT and pool were not measured together. Raster-heavy pages still cost hundreds of milliseconds.

Names and marks: ideas, not decisions

The project name is still undecided. A rename to editPDF was considered and cancelled because editpdf.io exists. Everything below is exploration.

Names already in the repository

NAME	WHERE IT APPEARS TODAY	STATUS
csharp-pdf	Repository IEvangelist/csharp-pdf and the Pages path /csharp-pdf/	Working name
PortableDocument	Assemblies and namespaces PortableDocument.*, and the static render API	In the API
AdoBye	tools/AdoBye.*, benchmark output and the editor header (“Ab” tile)	Legacy
editPDF	Considered as a rename, then cancelled: editpdf.io already exists	Cancelled

Six mark directions



Quire

Idea

A quire is a gathered set of sheets. The overlapping squares already appear on the sample covers.



Folio

Idea

A folded sheet with text lines. Reads as “document” at 16 px and carries a gradient well.



AdoBye

In use

The editor header today: an “Ab” tile on a blue gradient. A legacy codename, kept here for comparison.



Caret

Idea

Text lines and an amber insertion caret: the editor's core gesture of changing existing text.



Xref

Idea

Five linked nodes, the object graph and cross-reference table underneath every PDF.



csharp-pdf

In use

A sharp sign on a page. Literal and nerdy, and tied to the current repository name.

BEFORE ANY OF THIS BECOMES A DECISION

Unvetted: no trademark, domain, package-name or app-store search has been done for any name on this page. “In use” marks are shown for comparison, not as the recommendation.

Caret: name due diligence

If the project took the Caret direction, here is what a quick check on 2026-10-31 found. Not legal advice, and not a clearance: it is a first look.



Caret is the editor's core gesture: the insertion point where text changes. It is also the PDF annotation subtype /Subtype /Caret, so the name ties to the format itself. Both facts cut two ways, as below.

Name due diligence (2026-10-31, not legal advice)

WHERE	WHAT THE CHECK FOUND	READING
NuGet	Ids Caret and Caret.Pdf returned 404. Nearby ids such as WpfCaret exist.	Unclaimed
npm	caret is taken: a tiny package that inserts a caret and text beneath a string.	Taken
GitHub	A user or organisation named caret already exists.	Taken
caret.io	An established Markdown and text editor (Caret.io Ltd.; Mac, Windows, Linux; winget package Caret). Same broad category, text editing: the biggest trademark and discoverability risk.	Risk
caretpdf.com	Does not resolve. That is not proof it is available; WHOIS has not been checked.	Unverified
getcaret.com	Resolves, so presumably taken.	Taken
Search noise	caret is a well-known R machine-learning package, and /Subtype /Caret is a PDF annotation name. That hurts search, but also ties to the insertion-caret mark.	Noise

Recommended next steps

- Run a formal trademark search (USPTO and EUIPO), with counsel, focused on software and text-editing classes because of caret.io.
- Check domain availability with WHOIS before relying on caretpdf.com or any variant.
- Reserve the NuGet package ids (Caret, Caret.Pdf) and the npm scope, and consider a GitHub organisation, if the name is chosen.
- Weigh the alternatives on the previous page: a compound such as Caret.Pdf may separate it from caret.io, but that is a legal question, not a design one.

STATUS OF THIS CHECK

Checked by hand on 2026-10-31 against public registries and sites. Results can change; nothing here is a trademark opinion, and no name has been chosen.

Palettes, with measured contrast

Four palettes. Each swatch shows its WCAG 2.x contrast against white and against ink (#0F172A), computed by this generator.

Ink and Indigo

The design system of the committed samples and of this document.

In use: samples 01–27

Ink	Indigo deep	Indigo	Violet	Teal	Amber
#0F172A	#312E81	#4F46E5	#7C3AED	#0D9488	#F59E0B
White 17.85:1 AAA	White 11.42:1 AAA	White 6.29:1 AA	White 5.70:1 AA	White 3.74:1 large only	White 2.15:1 fails
Ink 1.00:1 fails	Ink 1.56:1 fails	Ink 2.84:1 fails	Ink 3.13:1 large only	Ink 4.77:1 AA	Ink 8.31:1 AAA

Editor midnight

Dark UI tokens from editor.css and the “Ab” logo gradient.

In use: web editor

Background	Raised	Accent	Accent deep	Text	Success
#0E1013	#1B1E25	#4C8DFF	#2F5FD0	#E8EAF0	#3ECF8E
White 19.05:1 AAA	White 16.68:1 AAA	White 3.20:1 large only	White 5.72:1 AA	White 1.20:1 fails	White 2.00:1 fails
Ink 1.07:1 fails	Ink 1.07:1 fails	Ink 5.58:1 AA	Ink 3.12:1 large only	Ink 14.84:1 AAA	Ink 8.94:1 AAA

Vermilion and paper

Warm editorial: ink on paper with one hot accent.

Exploratory

Ink	Paper	Vermilion	Rust	Deep teal	Gold
#1C1917	#F5F0E6	#C2410C	#9A3412	#115E59	#CA8A04
White 17.49:1 AAA	White 1.14:1 fails	White 5.18:1 AA	White 7.31:1 AAA	White 7.58:1 AAA	White 2.94:1 fails
Ink 1.02:1 fails	Ink 15.72:1 AAA	Ink 3.45:1 large only	Ink 2.44:1 fails	Ink 2.35:1 fails	Ink 6.08:1 AA

Aurora

Dark, luminous: a night background with four neon-soft accents.

Exploratory

Night	Indigo night	Mint	Lavender	Pink	Sun
#0B1020	#1E1B4B	#5EEAD4	#A78BFA	#F472B6	#FBBF24
White 18.93:1 AAA	White 15.99:1 AAA	White 1.48:1 fails	White 2.72:1 fails	White 2.65:1 fails	White 1.67:1 fails
Ink 1.06:1 fails	Ink 1.12:1 fails	Ink 12.07:1 AAA	Ink 6.56:1 AA	Ink 6.74:1 AA	Ink 10.69:1 AAA

Gradient ramps



White text needs 4.5:1 (AA) for body text, 3:1 for large text and graphics, 7:1 for AAA. Teal #0D9488 reaches only the large-text bar and amber #F59E0B fails with white, so this document uses darker tints (#0F766E, #B45309) for any teal or amber text.

Text pairings that pass

Aa 17.06:1	Aa 6.29:1	Aa 11.42:1	Aa 12.80:1	Aa 8.31:1	Aa 5.18:1
Ink on paper	White on indigo	White on indigo deep	Night on mint	Ink on amber	White on vermilion

Type pairings from fonts we already ship

Only faces whose licences are already in samples/_gen/fonts: Source Serif 4, Source Sans 3, Source Code Pro (SIL OFL 1.1) and Bitstream Vera Sans.

Pages that read well

Display in serif, text in a humanist sans.

A tagged PDF keeps its reading order even when the drawing order is different. That is the reason to structure it.

EDITORIAL

Source Serif 4 Semibold + Source Sans 3 (the sample system)

Aa Bb Cc 0123 → × ±

Aa g *R*

Open, edit, save

One family, clear weights, code in mono.

Call PdfEditor.Open(bytes), change a page, then Save with object streams.

```
var pdf = PdfEditor.Open(bytes);
```

PRODUCT

Source Sans 3 Semibold + Regular + Source Code Pro

Aa Bb Cc 0123 → × ±

Aa g R

A report in one voice

Serif throughout, with italic for emphasis.

Long reading likes a quiet serif. Headings carry weight, the body stays calm, and figures speak in captions.

BOOKISH

Source Serif 4 Semibold + Regular + Italic

Aa Bb Cc 0123 → × ±

Aa g *R*

Plain and sturdy

A compact TrueType pair for dense tables.

Vera is wide and open, which helps at small sizes in dashboards and spec tables.

UTILITY

Bitstream Vera Sans Bold + Regular (TrueType)

Aa Bb Cc 0123 → × ±

Aa g R

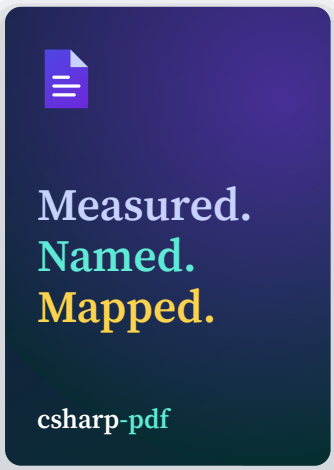
Scale used in this document

Aa	Aa	Aa	Aa	Aa	Aa	Aa	Aa
7.8 pt	8.6 pt	9.2 pt	12 pt	13 pt	16 pt	25 pt	64 pt

Inter and JetBrains Mono appear in the editor's CSS stack but are not shipped in this repository, so they are not proposed. Licences: samples/_gen/fonts/LICENSES.txt and bitstream-vera-license.txt.

Mock applications

The Folio mark and the Ink and Indigo palette on surfaces a project actually needs. The wordmark uses today's repository name; any name from the previous pages drops in.



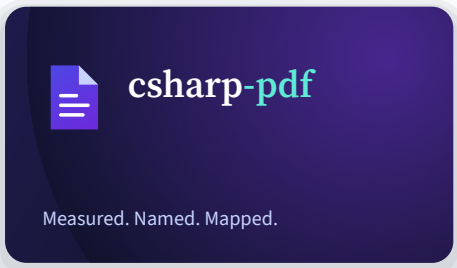
REPORT COVER



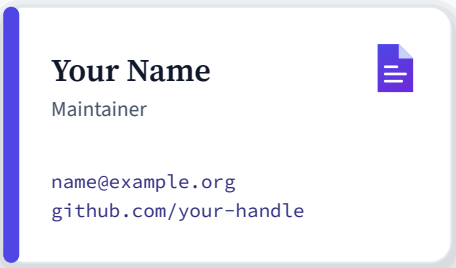
LETTERHEAD



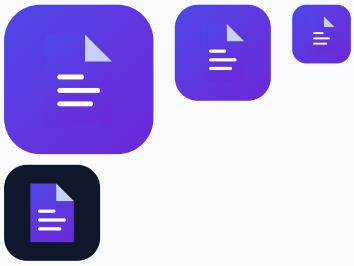
SOCIAL CARD 1200 × 630



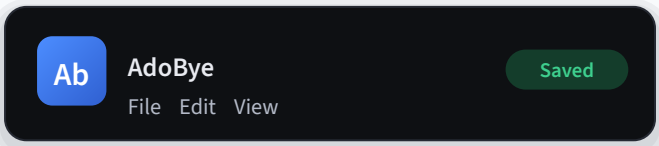
BUSINESS CARD, FRONT



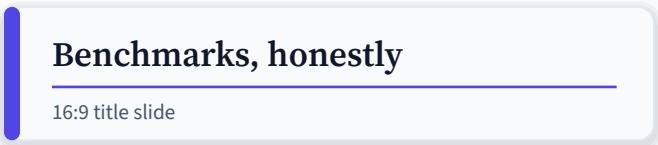
BACK



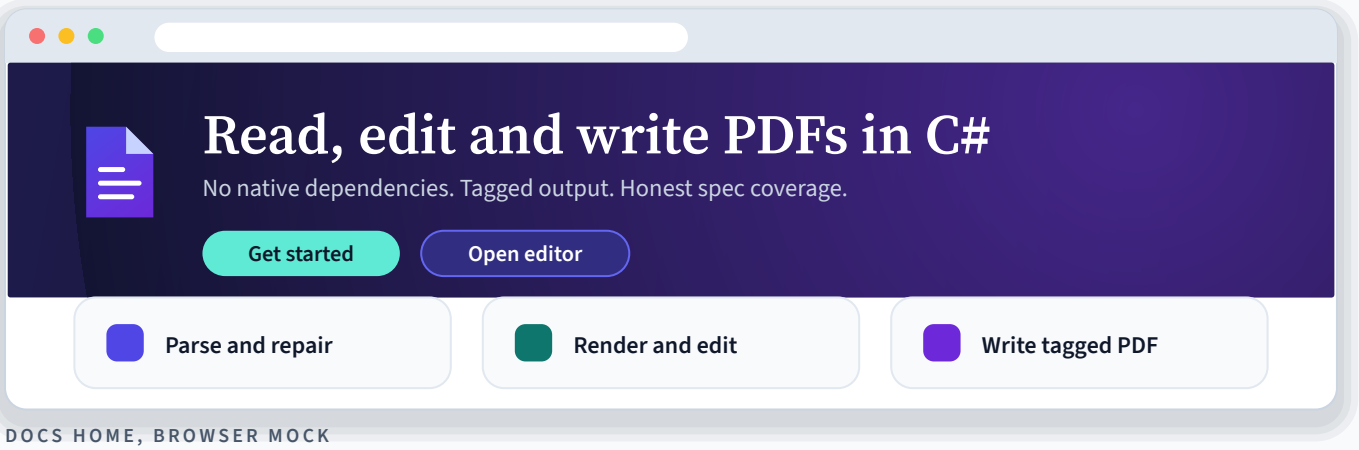
APP ICON, 56 / 36 / 22 AND DARK



EDITOR HEADER (CURRENT NAME AND MARK)



TITLE SLIDE



DOCS HOME, BROWSER MOCK

ISO 32000-2: where parity stands

Full parity is the goal. This is the honest state: 15 Partial rows, 0 Todo rows, and 2 Done rows that carry stated limits. Nothing on these pages is a conformance claim.

Rows by status

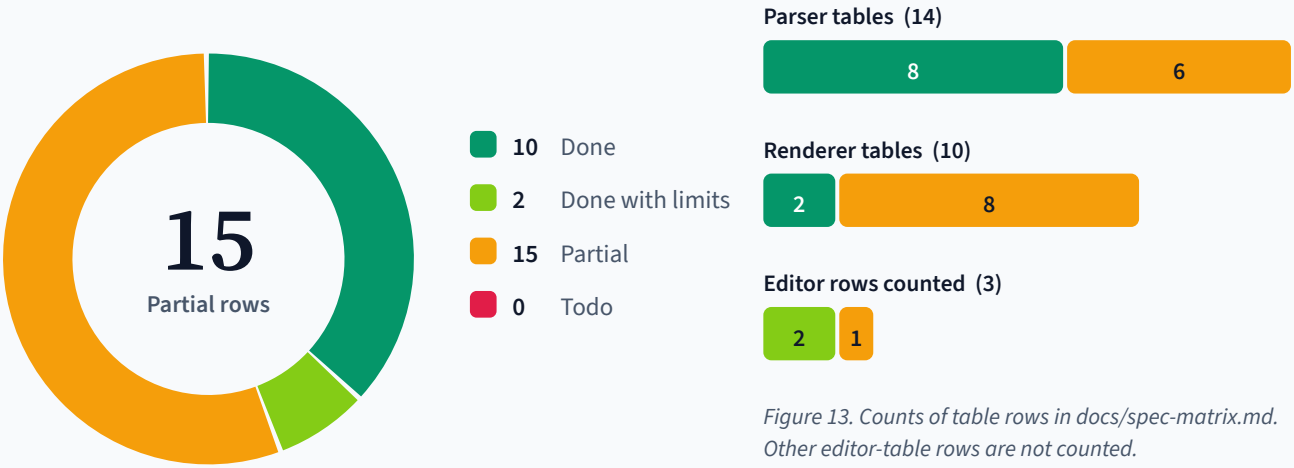


Figure 13. Counts of table rows in docs/spec-matrix.md. Other editor-table rows are not counted.

How strong is the evidence behind a status?

Independent reference

- PDFium: selected shading and rendering fixtures compared clean.
- OpenJPEG 2.5.4: JPX packed-header fixtures match exactly (384 samples each).
- libjpeg-turbo 3.2.0: 12-bit arithmetic DCT, maximum error 3, default conditioning only.
- Pillow 12.1.1 and LittleCMS 2.17: ICC within two 8-bit levels.
- pikepdf and pyHanko: encryption and signature interoperability.

Self-derived or analytic

- JPX subsampling fixtures are self-derived.
- Some overprint tests are self-consistent only.
- Timestamp unit tokens are self-built; no live TSA.
- Public-key encryption is round-trip tested only.
- U3D and PRC fixtures are synthetic.
- Page-group CMYK and ICC blending: analytic tests.

Not run, or unreliable

- veraPDF was not run; PDF/A and PDF/UA rules were compared by hand.
- New page-group blending was not compared with Chrome or PDFium.
- CJK pixel parity: the Edge comparison was unreliable.
- Non-byte-aligned JBIG2 widths and nonzero RDW/RDH: no reference.
- Non-default 12-bit DAC: untested.

Landed recently; no Partial row was promoted

ICC v4 black point, B2A tables · Coons and tensor patches · Arbitrary-path redaction · Annotation authoring API · AcroForm authoring and flatten · Fallback fonts, CJK chunks · DeviceCMYK table · Page-level undo and redo · Worker-pool band rendering · Inert U3D, PRC, RichMedia models

SOURCE
docs/spec-matrix.md and docs/spec-backlog.md

RUN
read at aa127037

HARDWARE
Not a measurement: a count of status words in two markdown tables.

MODE Status words are the repository's own. Counted: the parser table (14 rows), the renderer table (10 rows), and the one Partial and two limited-Done rows of the editor table. Test counts are recorded per checkpoint in the matrix and are not restated here.

Parser and file structure

Clause by clause, in the order of the matrix. Done means implemented and covered by tests, never that every corner of the clause is proven.

CLAUSE	AREA	STATUS	HONEST CAVEAT
7.2–7.3	**Lexical conventions, all object types**	Done	Tolerant numbers, #xx names, literal and hex strings.
7.5	**File structure: xref tables and streams, hybrid, Prev chain, object streams**	Done	Read and written, including object streams.
7.5.6	**Recovery of damaged files**	Done	Scans for “n g obj”; covered by the RepairsBrokenXref test.
7.4	**Filters: Flate, LZW, ASCIIHex, ASCII85, RunLength, predictors**	Done	Image codecs are the next row.
7.4	**Image codecs: DCT, CCITT, JBIG2, JPX**	Partial	Gaps: SOF11 arithmetic lossless, hierarchical JPEG, subsampled interleaved lossless. JBIG2 non-byte-aligned symbol widths and nonzero RDW/RDH, and non-default 12-bit DAC, lack independent references.
7.6	**Encryption: Standard RC4/AES, public-key**	Done	Standard R2–R6 read and write (pikepdf-verified). Public-key handler is round-trip tested only; no third-party validator.
7.7	**Document structure: page tree, inheritance**	Done	No caveats recorded.
7.8–7.9	**Content streams, resources, functions (types 0, 2, 3, 4)**	Done	No caveats recorded.
7.10–7.11	**File specifications, embedded files, data structures**	Partial	Remaining: broader tree-boundary, attachment and preservation combinations. /FS, /Thumb, /EP, /RF and /CI stay inert; external payloads are never opened.
12	**Annotations, forms, outlines, actions**	Partial	Typed inert action models; nothing executes. Open: exhaustive action-bearing objects and triggers, annotation appearances, complete form and action semantics.
14	**Metadata, tagged PDF, logical structure**	Partial	Open: external streams, arbitrary owner graphs, per-invocation tagging. Plain ExtractText excludes annotation text. Diagnostics are not PDF/UA certification.
14.8	**PDF/A and PDF/UA validation**	Partial	Heuristic validators. Rules were compared by hand against published veraPDF profiles; veraPDF was not run. Not certification.
Robustness	**Fuzz and truncation tests**	Done	Deterministic mutation of all fixtures (400-seed run green); caps on lexer nesting and inflate size.
Perf	**Hot paths**	Partial	Tuned rasteriser, JPEG and caches. Fresh-vector and text CPU gains were not reliable (see the benchmark pages).

Source: docs/spec-matrix.md, Parser table, read at commit aa127037. Caveats are condensed; the matrix and docs/spec-backlog.md hold the full text.

Renderer and editor

Eight of ten renderer rows are Partial. The editor's two Done rows are bounded APIs, marked Done because their limits are part of the status.*

CLAUSE	AREA	STATUS	HONEST CAVEAT
8.4	**Graphics state, paths, clipping**	Done	Anti-aliased scanline raster; non-zero and even-odd.
8.4.3	**Stroking: width, caps, joins, miter, dash**	Done	No caveats recorded.
8.6	**Device, Cal*, Lab, Indexed, Separation, DeviceN, ICCBased**	Partial	ICC matrix/TRC and LUT to sRGB; LittleCMS-compatible black-point compensation is a compatibility choice, not ISO 18619. D2B/B2D, device-link, general CMM and gamut mapping are open.
8.7	**Tiling and shading patterns**	Partial	Types 1–7 including Coons and tensor surfaces. Tessellation is finite and bounded; broader function and colour-space combinations and independent fixtures are open.
8.9–8.10	**Images, inline images, XObject forms**	Partial	Stencil, colour-key and soft-mask alpha work; completeness depends on the codec gaps above.
9	**Fonts: TrueType, Type0/CID, Type3, ToUnicode**	Partial	CFF, Type 1 (incl. Multiple Master) and all 59 predefined CMaps. CJK fallback is covered by fixtures, not pixel parity; serif CJK reuses the sans design.
11	**Transparency: alpha, blend modes**	Partial	Alpha, 16 blend modes, groups and soft masks are Done. CMYK/Cal/ICC page-group blending is analytic-tested, not compared with Chrome or PDFium.
8.11	**Optional content (OCG/OCMD)**	Partial	Visibility expressions, configurations, locks, radio groups. State changes are in memory; language validation is structural only.
10	**Rendering intents, overprint, halftones, transfer**	Partial	Overprint simulation is opt-in and RGB-approximate; halftone, flatness and smoothness remain inert.
13	**Multimedia, 3D, RichMedia, XFA, JavaScript**	Partial	Typed inert metadata only; nothing executes or plays. U3D and PRC fixtures are synthetic.
Editor	**Moving and resizing existing images and vector objects**	Partial	Page-level and uniquely invoked Form children. Shared, repeated or nested Form children are withheld; complex clip ranges unsupported.
Editor	**Redaction (Redactor.Apply)**	Done*	Done for this bounded API, not universal sanitisation: full rewrite, whole-object fallback, glyph and outline-text limits.
12.8	**Digital signatures**	Done*	Detached SHA-256, DocMDP, RFC 3161 and DSS reading. Chain trust, revocation, PAdES LTV and DSS authoring are unsupported.

Done = Done for the stated bounded scope only. Source: docs/spec-matrix.md (Renderer and Editor tables), read at commit aa127037.*

Method and provenance

Where each figure came from, how this file was made, and what it cannot tell you.

Sources

FILE	USED FOR	VERSION
bench/AdoBye.Bench/results/2026-10-30.md	BenchmarkDotNet tables: open, extract, render stages	2026-10-30
bench/AdoBye.Bench/README.md	Corpus, preflight, measurement boundaries	2026-10-30
docs/performance/2026-10-30.md	Editor before/after, heartbeat, budgets	2026-10-30
docs/performance/2026-10-30-aot.md	JIT versus AOT, download size, goal misses	2026-10-30
docs/performance/2026-10-30-aot-budgets.json	The six strict-goal misses	2026-10-30
docs/performance/2026-10-30-worker-pool.md	Worker pool and band rendering	2026-10-30
docs/spec-matrix.md, docs/spec-backlog.md	Parity rows and caveats	aa127037
tools/AdoBye.EditorWeb/wwwroot (editor.css, index.html)	Editor palette and “Ab” mark	aa127037
samples/_gen/fonts/LICENSES.txt	Font licences	aa127037

How this file was made

- Written by tools/AdoBye.Samples (Gen28) with this repository's own lexer, writer and embedded-font subsetter. Charts are vector paths, shadings and transparency; there are no images and no external references.
- Tagged: a structure tree with headings, paragraphs, lists, tables with header scope, figures with alternative text and artifacts for page furniture; document language, title, outline and tab order are set.
- Fonts embedded as subsets: Source Serif 4, Source Sans 3, Source Code Pro (SIL OFL 1.1) and Bitstream Vera Sans. Regeneration is byte-identical.
- Rendered with this repository's renderer and compared with Chromium through tools/AdoBye.RenderCompare during development.

What this document cannot tell you

LIMITS

Every benchmark comes from one development machine (Windows 11, Core i9-10900) and from reports dated 2026-10-30. Results do not transfer to other hardware, corpora or runtimes.

The PDF/UA check used here is this repository's heuristic validator. It is not veraPDF, not Matterhorn, and not certification.

The brand pages are design exploration. No name, mark or palette has been chosen, vetted or approved.

Parity percentages are deliberately not given: rows are not equally large, and Partial rows differ widely in how much remains.

REGENERATE

```
dotnet run --project tools/AdoBye.Samples -- generate samples 28-
```